Sequence BIO TAGGING

**Task 1 - Long Short-Term Memory (LSTM)**

Pre Processing Steps =>
a) Strip whitespace
b) Replace all numbers (integers/floats) with <num>
c) Convert to lowercase
d) Add special <pad> label for sequence padding
e) Extract all unique BIO labels from training data
f) Created a pickle file for GloVe embeddings to increase efficient from the Stanford NLP .vec file
g) use the .bin file for fasttext embeddings with the inbuilt fasttext module,helped in dealing with the embeddings with great ease

Final Model HyperParameter List =>

| Hyperparameter | Value | Description |
|---|---|---|
| k (hidden_size) | 15 | Dimension of the Hidden Layer Vector. Basically model width. |
| BATCH_SIZE | 32 | Training batch size |
| EPOCHS | 12 | Total training epochs |
| LR | 1e-3 (0.001) | Adam learning rate |
| MAX_VOCAB | 200,000 | Max vocabulary size |
| MIN_FREQ | 1 | Minimum token frequency threshold |
| EMB_DIM | 300 | Fasttext Embd |
| LSTM layers | 2 | Number of stacked LSTM layers. (Basically model |
| | | depth) |
| Bidirectional | True | Use BiLSTM |

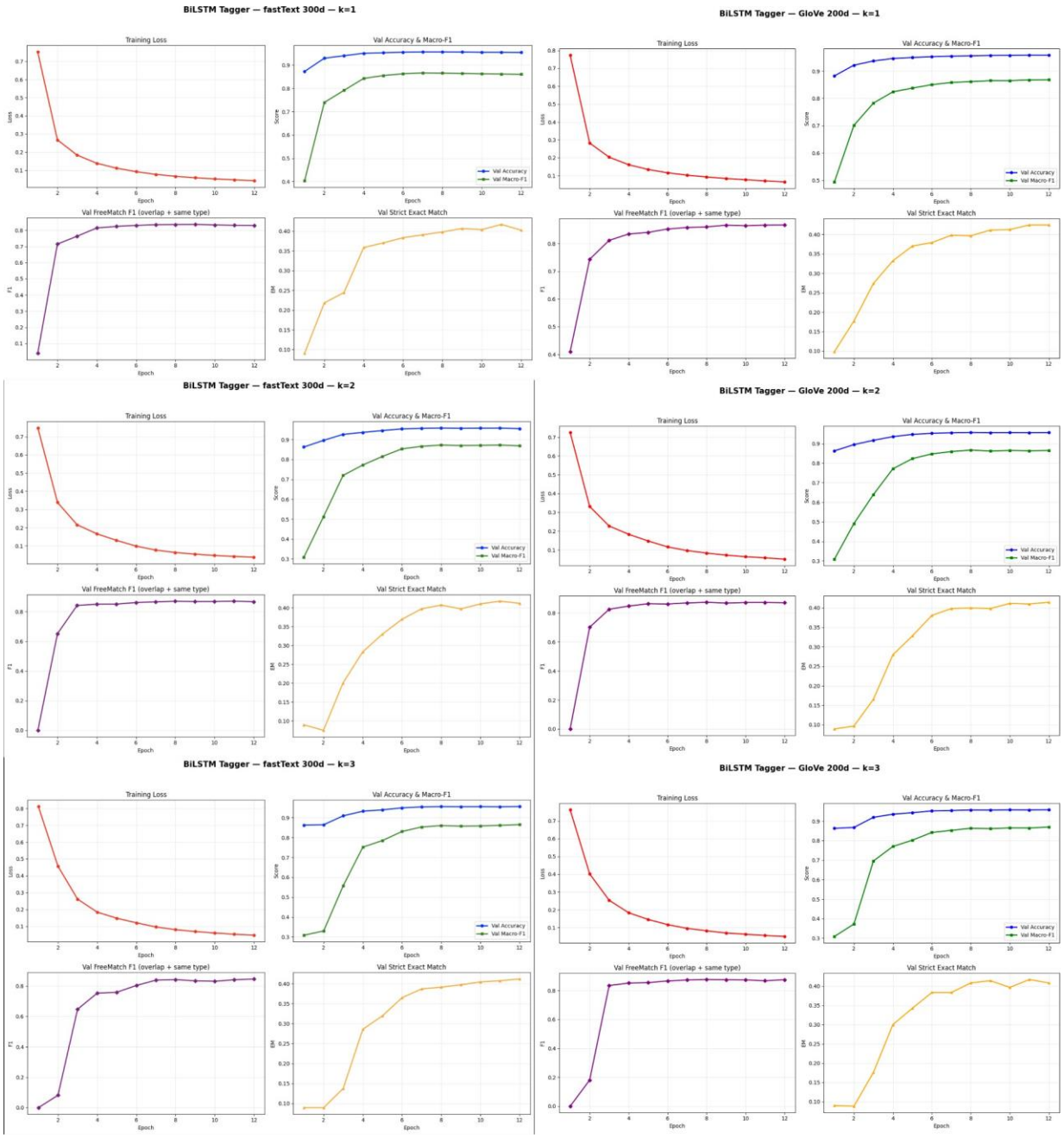
| | | |
|---|---|---|
| Gradient clip | 5.0 | Max gradient norm |
| Dropout | 0.0 | No dropout (not implemented) |

Training Setup =>
a) CrossEntropyLoss with ignore_index=ltoi["<pad>"] - Ignores padded positions during loss computation.
b) Adam with learning rate 1e-3
c) Default Initial Weights
d) Gradient Clipping - to prevent vanishing gradient and exploding gradient.
e) No dropout, saw no need for regularization as val acc was decent

Final Model Architecture =>
a) Use of fastext embeddings with 2 LSTMs stacked over each other working with 30D vectors and 12 epochs training iterations - gave the best results among the 6 experiments that we conducted.
b) Layer 1 BiLSTM:
    Forward LSTM: 300D → 15D
    Backward LSTM: 300D → 15D
    Concatenated: 30D
c) Layer 2 BiLSTM:
    Forward LSTM: 30D → 15D
    Backward LSTM: 30D → 15D
    Concatenated: 30D
d) Linear Layer : 30D => Number of Labels, Produces one prediction per token

Results (Plots are also present in the artifacts folder) (Results are not reproducible) =>

| Number of Layers | Glove FreeMatch F1 | Glove Exact Match EM | Fasttext FreeMatch F1 | Fasttext Exact Match |
|---|---|---|---|---|
| 1 | 0.8667 | 0.4246 | 0.8674 | 0.4070 |
| 2 | 0.8703 | 0.4275 | 0.8671 | 0.4348 |
| 3 | 0.8744 | 0.4173 | 0.8662 | 0.4202 |




Explanation ExactEM match metric to define better embeddings=>
The fasttext model performs better because of the way it fundamentally builds embeddings.
Now, fasttext uses the ngrams model and the embedding of the word is a component-wise sum of the embeddings of the respective individual ngrams.

Now => this helps us still get a relatively decent embedding for an OOV word as some of its ngrams must be present and we can have its learned embeddings, hence resulting in a logical embedding for an OOV word, whereas in case of GloVe a word is an atomic unit, if a word is not present then its embedding is initialised to <unk> and if lot of words ot present all start with same and tough to make them reach their desired embedding. Hence, it outperforms, however in our case

GloVe : 200 length

Fasttext : 300 length

Fastetxt might be only slightly better as bigger embedding size is increasing complexity (Just a hypothesis). In theory fasttext will always be better.

Some key analysis of the results =>

1) Increasing k always results in increasing the strict match EM and the free match F1, however till a point after that it stagnates and might result in overfitting.

2) Increasing epochs also follow the same pattern, stagnating a little earlier than k.

Also one of the key things is that => results keep on changing across runs, but this is the general trend we noticed.

Prompts Used =>

a) Where can i find pre trained embeddings

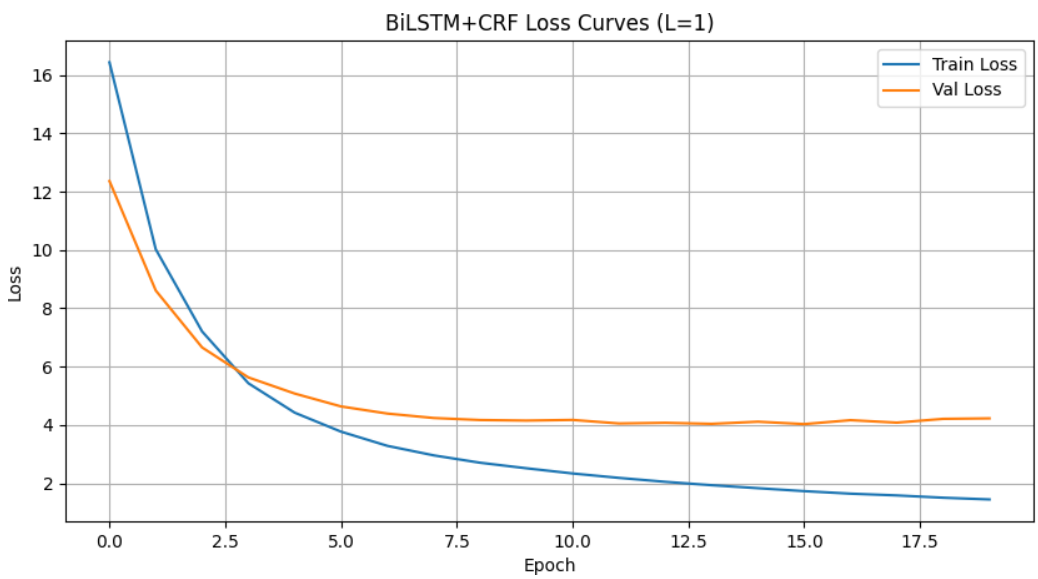
b) Write simple pytorch code for BiLISTM model

c) Write a custom training loop using the above BiLISTM model, also implement the FreeF1 match and Strict Match EM metrics

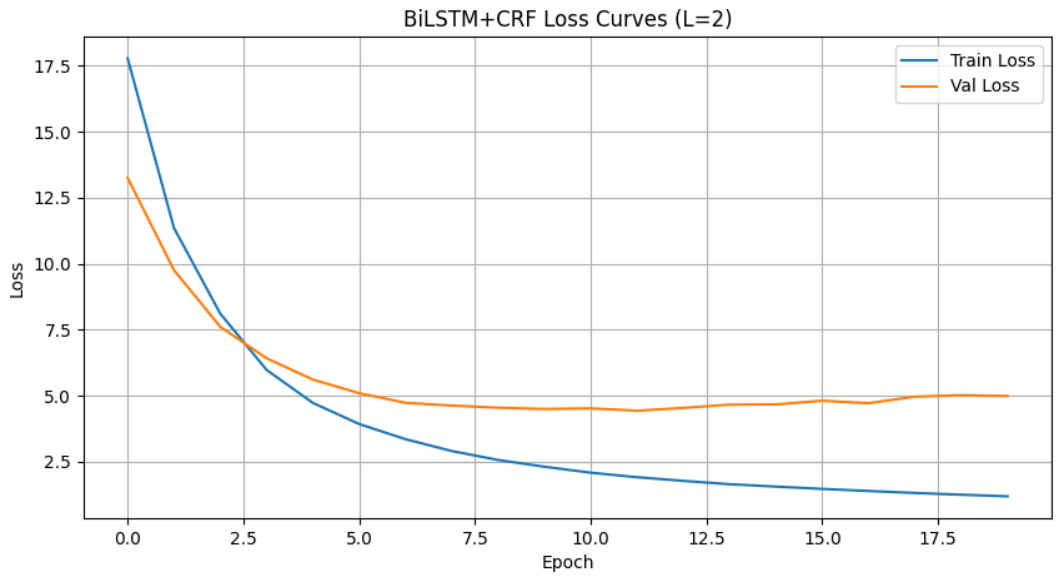
d) How to check quality of embeddings

**Task 2 - BiLSTM+CRF**

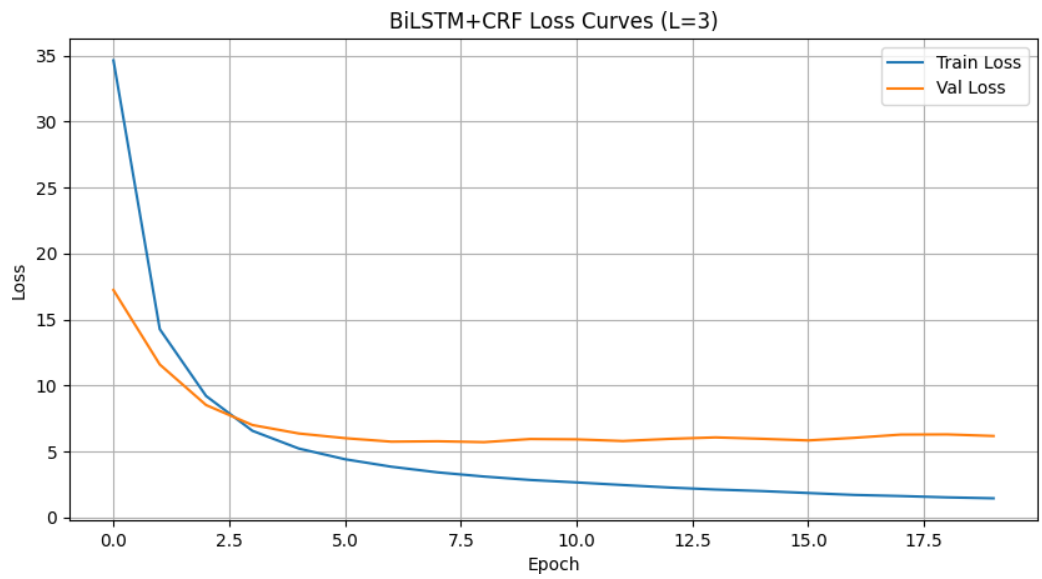
1. **Short description**
    a. Embedding layer: Pre trained FastText embeddings initialized from a matrix
    b. Stacked BiLSTM Encoder: Concatenates hidden states from both directions at each timestamp, dropout is applied between the hidden layers(forL>1).
    c. Linear Projection Layer: Reduces emission scores for each tag at each position, output shape is (batch_size, seq_len, num_tags)
    d. CRF Layer: Forward algo computes the partition function and viterbi decoding finds the most likely tag sequence during inference. Loss function is -log(partition-gold score)
    e. Hyperparameters:
    hidden_dim = 64, dropout_rate = 0(during training, keeping dropout was surprisingly increasing overfitting, so we decided not to keep it), learning rate=0.0001(for smooth gradient descent), batch_size=32, num_epochs=20, embedding_dim=300, weight_decay=1e-4(for regularization)
    f. During training, each batch of token sequences passes through the embedding layer, stacked BiLSTM encoder, and linear projection to generate emission scores, which are then fed to the CRF layer for loss computation.
    g. Evaluation is performed on the validation dataset with the model in evaluation mode, which disables dropout and gradient computation.
2. **Plots and metrics**



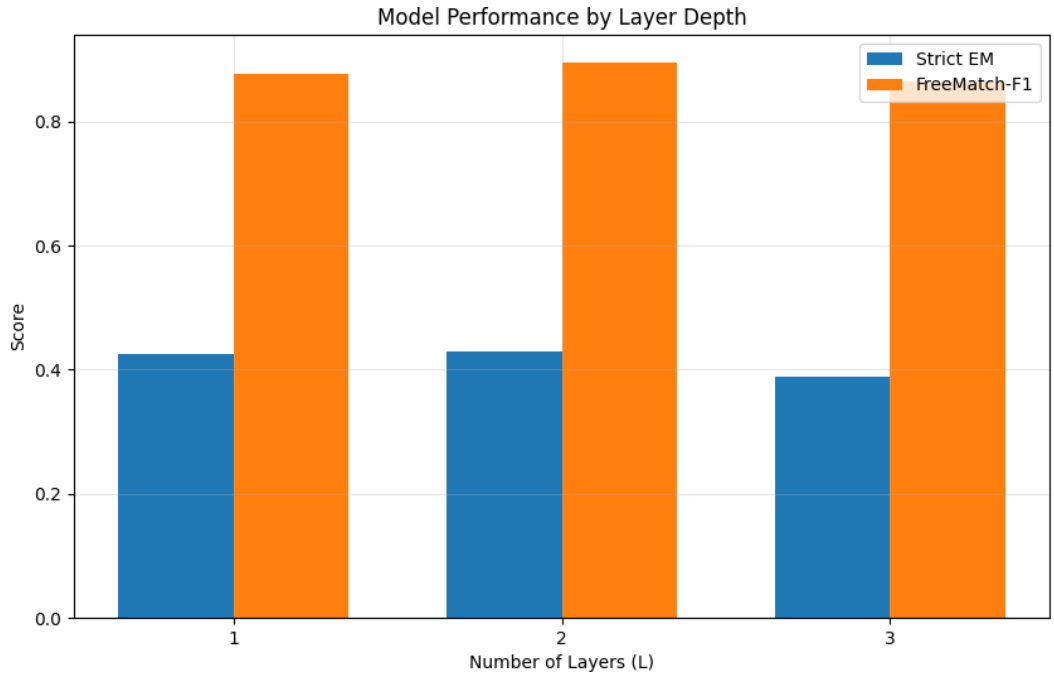

a.


BiLSTM+CRF Loss Curves (L=2)
Train Loss
Val Loss
Loss
Epoch
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

b.


BiLSTM+CRF Loss Curves (L=3)
Train Loss
Val Loss
Loss
Epoch
35
30
25
20
15
10
5
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

c.




d.

e. Metrics

```
L=1 Results:
  Strict EM: 0.4246
  FreeMatch-F1: 0.8768
  Precision: 0.9302
  Recall: 0.8291
Evaluating: 100%|██████████| 22/22 [00:01<00:00, 12.40it/s]

L=2 Results:
  Strict EM: 0.4290
  FreeMatch-F1: 0.8952
  Precision: 0.9315
  Recall: 0.8617
Evaluating: 100%|██████████| 22/22 [00:01<00:00, 11.36it/s]

L=3 Results:
  Strict EM: 0.3895
  FreeMatch-F1: 0.8646
  Precision: 0.9150
  Recall: 0.8194
```

## 3. Comments

a. Increasing the number of layers/depth and thus the model complexity may cause overfitting as we see after L2.

b. Initially the number of neurons per hidden layer were taken to be 256 which caused immense overfitting despite using dropout with rate 0.8. 64 was found to be a suitable number of neurons in the hidden layer.

c. Although the code allowed for dropout, I did not use it since there was degradation in performance.
d. The train loss curve initially was very sharp which was reflected by the poor validation loss due to overfitting.
e. To counter this, the learning rate was set to 1e-4. Along with this, batch SGD was used to serve as the middle point between SGD and BGD. This led to a smooth loss vs epoch curve.
f. Early stopping was not used at the moment for the sake of graph plots.
g. Weighted regularization penalty with weight_decay=1e-4 also aided in reducing the amount of overfitting.
h. The best model is for L=2.
i. Strict EM requires the entire predicted tag sequence to exactly match the ground truth sequence whereas FreeMatch-F1 operates at the entity span level, giving partial credit for partially correct predictions.
j. The model has high F1 but low EM indicating it identifies most entities correctly but makes frequent minor errors in boundary detection or has isolated tagging mistakes.

**4. Major Prompts used(apart from general debugging)**

a. Find the issue in this code causing the loss to become negative due to the gold score exceeding partition value.
b. Give me the code for testing the evaluation metrics of Strict EM score and Free-Match F1 assuming the models are saved.
c. What are some suitable hyperparameters to train the models in the following context?
d. Give a code cell to test the validity of padding in the following code.
e. Give a code cell to find the instances when the value of gold_score-partition<0
f. Give a code cell to test the implementation of CRF in this code.

**Task 3 - BiLSTM+Bahdanau Attention**

1) Firstly we build vocabulary both for train and val sets while doing this we also lowe case the token too ,we do this because we need ID for words after this step we have something like:

```
{
    "<PAD>": 0,
    "<UNK>": 1,
    "delhi": 2,
    "court": 3,
    "rajiv": 4,
     so on
}
```

basically this does unique word -> unique ID

2) Preprocessor : here i have data like this currently:

```
[
  {
    "id": "Sentence #1989",
    "tokens": ["``", "Delhi", "Supreme", "Court", "terms"],
    "labels": ["O", "O", "O", "O", "O"]
  },
  {
    "id": "Sentence #496",
    "tokens": ["Anguished", "by", "the", "loss", "of"],
    "labels": ["O", "O", "O", "O", "O"]
  }
]
```

Now from step 1 i got a id mapping for the tokens similarly i have token maapping for labels too so basically i merge them in preprocessing step and return that thats the use of this function.

3) pretrained embedding : glove 200d

After loading glove["delhi"] → numpy array of shape (200,) and matrix dimension will be |vocab| x 200 this is basically my embedding weight matrix

Now we fill the vector for each word using this to handle OOV we randomly assign a vector to it

For case of UNK instead of random i am using the mean of all the vectors present.

Training setup:
parameters passed: as given we pass the number of train layers which will be 1,2,3 the embed matrix we got from previous steps, the length of vocab and train/test sets

Train_one function trains one bilstm+bahdanau attention for given L layers which will be 1,2 or 3
We use adam optimizer with cross entropy loss please note we ignore pad labels again because its not our data so it should not change results anyways
We track the best F1 score and return the model which provides the best results!

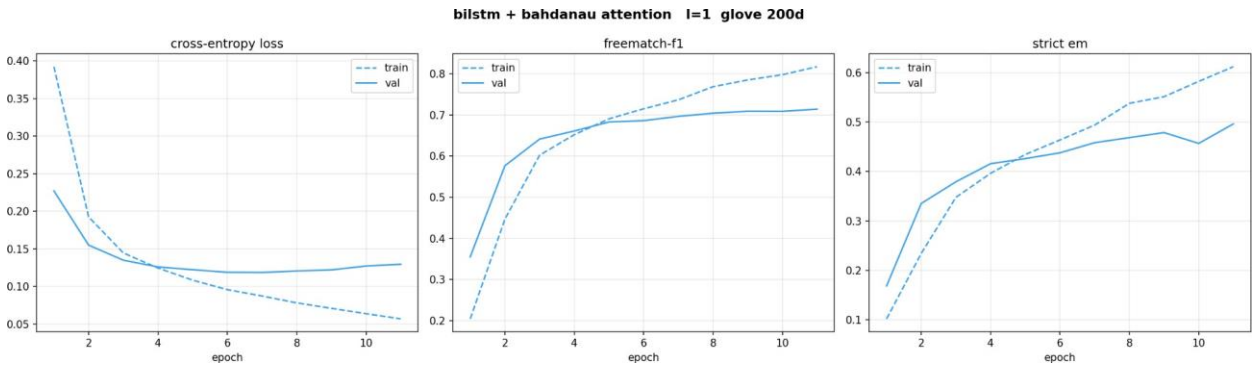
| Hyperparameter | Value |
|---|---|
| Embedding type | GloVe 200d |
| Embedding dimension | 200 |
| Hidden dimension | 96 |
| dropout | 0.3 |
| Optimizer learning rate | Adam, 0.001 |
| Lr scheduler | ReduceLROnPlateau (patience=3, factor=0.5) |
| batchsize | 32 |
| Epochs | 11 |
| Gradient clip | 5 |
| Loss | CrossEntropyLoss |

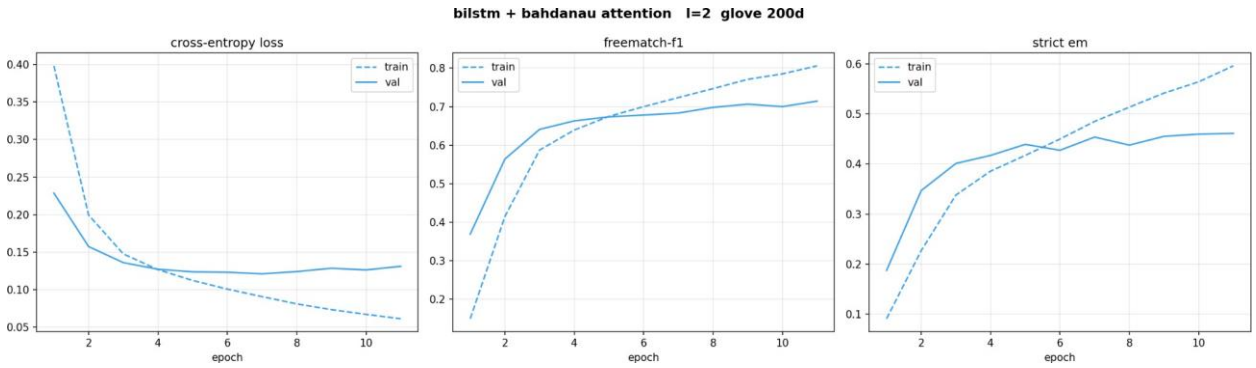
Evaluation:
*Please note i have done entity level F1 and not token level.
extractspan function i am getting something like-> { (1, 2, "LOC"), (4, 5, "LOC") }

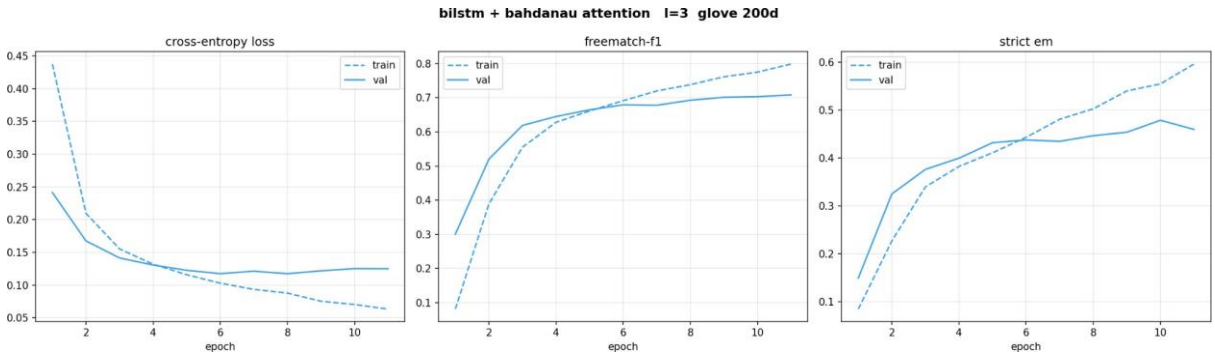
Results:

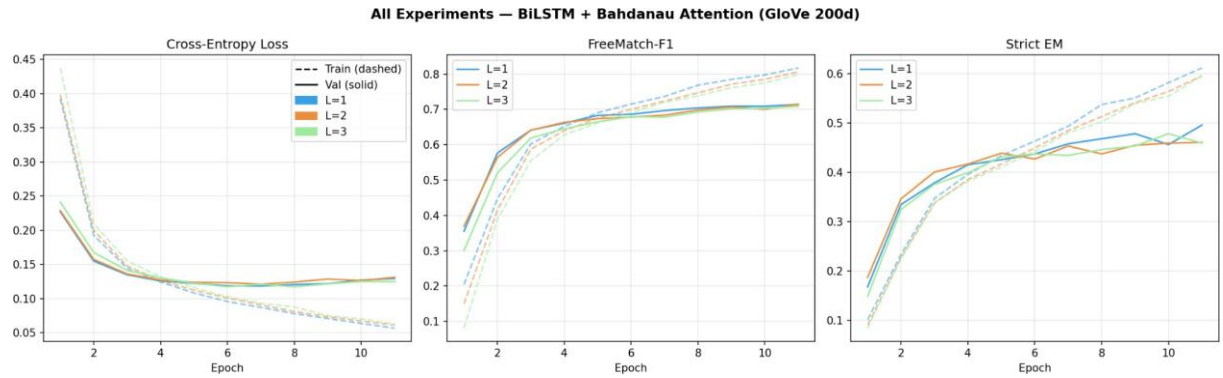
**Results — BiLSTM + Bahdanau Attention (GloVe 200d)**

| Layers (L) | FreeMatch-F1 | Strict EM | Precision | Recall |
|---|---|---|---|---|
| 1 | 0.7233 | 0.5139 | 0.7233 | 0.7233 |
| 2 | 0.7213 | 0.5022 | 0.7159 | 0.7267 |
| 3 ★ | 0.7264 | 0.5037 | 0.7243 | 0.7285 |






Best model, source code, pre trained embeddings are in the zip file.

Effects:

(d) Attention and Increasing Depth

1. increasing depth
   slightly improves freematch f1 (best at L=3 with 0.7264). deeper model capture richer contexts, but the gains are marginal it kind of saturated after 2 layers.
2. overfitting behavior
   training loss and F1 continue improving, while validation loss increases after early epochs. This means that overfitting happens as we grow layers.
3. Freematch f1 vs strict em Difference
   freematch f1 (≈0.72) is consistently higher than strict em (≈0.50).because F1 evaluates entity spans, while Strict EM requires the entire sentence labeling to be perfectly correct.a single token error causes EM to drop to zero.
4. bahdanau Attention
   attention mechanism allows each token to attend to all other tokens, improving modeling of multi token locations and long range dependency. This primarily benefits span level detection.
5. Best Configuration
   3 layer BiLSTM with bahdanau attention achieved the highest FreeMatch-F1, but improvements over L=1 were small. This suggests that attention contributes more to performance than simply increasing depth.